

```

1 import React, { useEffect, useState } from 'react'
2 import { StrictMode } from 'react'
3 import ReactDOM from 'react-dom/client'
4 import axios from 'axios'
5 import './index.css'
6
7 const SmartTravelDashboard = () => {
8   const [position, setPosition] = useState(null)
9   const [geoError, setGeoError] = useState(null)
10  const [gettingLocation, setGettingLocation] = useState(false)
11
12  const [weather, setWeather] = useState(null)
13  const [weatherLoading, setWeatherLoading] = useState(false)
14  const [weatherError, setWeatherError] = useState(null)
15
16  const [itinerary, setItinerary] = useState([
17    { id: '1', city: 'Paris', country: 'France', date: '2025-06-11', notes: 'Visit Louvre & Eiffel Tower' },
18    { id: '2', city: 'Rome', country: 'Italy', date: '2025-06-15', notes: 'Colosseum and Vatican Museums' },
19  ])
20  const [newDest, setNewDest] = useState({ city: '', country: '', date: '', notes: '' })
21
22  const [bookingJson, setBookingJson] = useState(null)
23
24  const [userData, setUserData] = useState({
25    name: 'Sabrina Graham',
26    email: 'Streaming@pl.biz',
27    phone: '+1-770-736-8031 x56442'
28  })
29  const [loadingUser, setLoadingUser] = useState(false)
30  const [userError, setUserError] = useState(null)
31
32  useEffect(() => {
33    getLocation()
34  }, [])
35
36  const getLocation = () => {
37    if (!navigator.geolocation) {
38      setGeoError('Geolocation not supported by your browser.')
39      return
40    }
41    setGettingLocation(true)
42    setGeoError(null)
43
44    navigator.geolocation.getCurrentPosition(
45      (pos) => {
46        const coords = {
47          lat: parseFloat(pos.coords.latitude.toFixed(4)),
48          lon: parseFloat(pos.coords.longitude.toFixed(4))
49        }
50        setPosition(coords)
51        setGettingLocation(false)
52      },
53      (err) => {
54        setGeoError(err.message || 'Unable to retrieve location.')
55        setGettingLocation(false)
56      }
57    )
58  }

```

```

56     },
57     { enableHighAccuracy: true, timeout: 15000 }
58   )
59 }
60
61 useEffect(() => {
62   if (position) {
63     fetchWeather()
64   }
65 }, [position])
66
67 const fetchWeather = async () => {
68   setWeatherLoading(true)
69   setWeatherError(null)
70   try {
71     const url = `https://api.open-meteo.com/v1/forecast?latitude=${position.lat}&longitude=${position.lon}&current_weather=true`
72     const response = await axios.get(url)
73     setWeather(response.data.current_weather)
74   } catch (error) {
75     setWeatherError('Failed to load weather.')
76   }
77   setWeatherLoading(false)
78 }
79
80 const addDestination = () => {
81   if (!newDest.city || !newDest.country || !newDest.date) return
82
83   const newDestination = {
84     id: Date.now().toString(),
85     city: newDest.city,
86     country: newDest.country,
87     date: newDest.date,
88     notes: newDest.notes || '',
89   }
90
91   setItinerary([...itinerary, newDestination])
92   setNewDest({ city: '', country: '', date: '', notes: '' })
93 }
94
95 const fetchUserData = async () => {
96   setLoadingUser(true)
97   setUserError(null)
98   try {
99     const response = await axios.get('https://jsonplaceholder.typicode.com/users/1')
100     setUserData({
101       name: response.data.name,
102       email: response.data.email,
103       phone: response.data.phone,
104     })
105   } catch (err) {
106     setUserError('Error fetching user data.')
107   }
108   setLoadingUser(false)

```

```

107     }
108     setLoadingUser(false)
109 }
110
111 const convertBookingXml = () => {
112     const xmlString = `
113         <booking>
114             <id>BK12345</id>
115             <traveler>
116                 <firstName>Alice</firstName>
117                 <lastName>Wong</lastName>
118             </traveler>
119             <flights>
120                 <flight>
121                     <from>NYC</from>
122                     <to>LHR</to>
123                     <date>2025-06-01</date>
124                 </flight>
125                 <flight>
126                     <from>LHR</from>
127                     <to>CDG</to>
128                     <date>2025-06-05</date>
129                 </flight>
130             </flights>
131             <hotel>
132                 <name>Grand Central Hotel</name>
133                 <nights>4</nights>
134             </hotel>
135         </booking>
136     `.trim()
137
138     const parser = new DOMParser()
139     const doc = parser.parseFromString(xmlString, 'application/xml')
140
141     if (doc.querySelector('parsererror')) {
142         setBookingJson({ error: 'Invalid XML' })
143         return
144     }
145
146     const convertNodeToJson = (node) => {
147         const obj = {}
148
149         for (const attr of Array.from(node.attributes)) {
150             obj[`${attr.name}`] = attr.value
151         }
152
153         const children = Array.from(node.children)
154         if (children.length === 0) {
155             return node.textContent?.trim() ?? ''
156         }
157
158         children.forEach((child) => {
159             const key = child.tagName
160             const value = convertNodeToJson(child)

```

```

160     const value = convertNodeToJson(child)
161     if (obj[key]) {
162         if (!Array.isArray(obj[key])) obj[key] = [obj[key]]
163         obj[key].push(value)
164     } else {
165         obj[key] = value
166     }
167     })
168     return obj
169 }
170
171 const root = doc.documentElement
172 const json = { [root.tagName]: convertNodeToJson(root) }
173 setBookingJson(json)
174 }
175
176 return (
177     <div className="dashboard-root">
178         <header className="header">
179             <h1>Smart Travel Companion Dashboard</h1>
180             <p className="tagline">Your interactive travel assistant</p>
181         </header>
182
183         <main className="grid">
184             <section className="panel media">
185                 <h2>Travel Media</h2>
186                 <div className="media-wrapper">
187                     <iframe src="https://www.youtube.com/embed/z7cRTSP0vwo"></iframe>
188                     <audio controls style={{ width: '100%', marginTop: '0.75rem' }}>
189                         <source src="https://www.w3schools.com/html/horse.mp3" type="audio/mpeg" />
190                         Your browser does not support audio.
191                     </audio>
192                 </div>
193             </section>
194
195             <section className="panel">
196                 <h2>Current Location</h2>
197                 {position && (
198                     <ul className="kv">
199                         <li>
200                             <span>Latitude:</span> <strong>{position.lat}</strong>
201                         </li>
202                         <li>
203                             <span>Longitude:</span> <strong>{position.lon}</strong>
204                         </li>
205                     </ul>
206                 )}
207                 {!position && !geoError && <p>{getLocation ? 'Detecting location...' : 'Location not set.'}</p>}
208                 {geoError && <p className="error">{geoError}</p>}
209                 <div style={{ marginTop: '.5rem' }}>
210                     <button onClick={getLocation} disabled={!getLocation}>
211                         {getLocation ? 'Getting location...' : 'Get Location'}

```

```

211 |         {gettingLocation ? 'Getting location...' : 'Get Location'}
212 |     </button>
213 | </div>
214 | <small>Uses HTML5 Geolocation API.</small>
215 | </section>
216 |
217 | <section className="panel">
218 |   <h2>Live Weather</h2>
219 |   {weatherLoading && <p>Loading weather...</p>}
220 |   {weatherError && <p className="error">{weatherError}</p>}
221 |   {weather && (
222 |     <div className="weather">
223 |       <div>
224 |         <span>Temperature:</span> <strong>{weather.temperature}°C</strong>
225 |       </div>
226 |       <div>
227 |         <span>Wind Speed:</span> <strong>{weather.windspeed} km/h</strong>
228 |       </div>
229 |       <div>
230 |         <span>Direction:</span> <strong>{weather.winddirection}°</strong>
231 |       </div>
232 |       <div>
233 |         <span>Time:</span> <strong>{weather.time}</strong>
234 |       </div>
235 |     </div>
236 |   )}
237 |   {!position && <p className="muted">Weather loads after location is found.</p>}
238 | </section>
239 |
240 | <section className="panel span-2">
241 |   <h2>Travel Plan (Dynamic JSON)</h2>
242 |   <pre className="json">{JSON.stringify(itinerary, null, 2)}</pre>
243 |
244 |   <div className="form">
245 |     <input
246 |       placeholder="City"
247 |       value={newDest.city}
248 |       onChange={e => setNewDest({ ...newDest, city: e.target.value })}
249 |     />
250 |     <input
251 |       placeholder="Country"
252 |       value={newDest.country}
253 |       onChange={e => setNewDest({ ...newDest, country: e.target.value })}
254 |     />
255 |     <input
256 |       type="date"
257 |       value={newDest.date}
258 |       onChange={e => setNewDest({ ...newDest, date: e.target.value })}
259 |     />
260 |     <input
261 |       placeholder="Notes"
262 |       value={newDest.notes}
263 |       onChange={e => setNewDest({ ...newDest, notes: e.target.value })}
264 |     />
265 |     <button onClick={addDestination} disabled={!newDest.city || !newDest.country || !newDest.date}>

```

```

    />
    <button onClick={addDestination} disabled={!newDest.city || !newDest.country || !newDest.date}>
      Add Destination
    </button>
  </div>
</div>
</section>

<section className="panel">
  <h2>User Data (Axios Fetch)</h2>
  <div>
    <p><strong>Name:</strong> {userData.name}</p>
    <p><strong>Email:</strong> {userData.email}</p>
    <p><strong>Phone:</strong> {userData.phone}</p>
  </div>
  {userError && <p className="error">{userError}</p>}
  <div style={{ marginTop: '.5rem' }}>
    <button onClick={fetchUserData} disabled={loadingUser}>
      {loadingUser ? 'Loading...' : 'Fetch Data'}
    </button>
  </div>
</section>

<section className="panel">
  <h2>Booking XML → JSON</h2>
  <button onClick={convertBookingXml}>Convert Booking XML</button>
  {bookingJson && <pre className="json">{JSON.stringify(bookingJson, null, 2)}</pre>}
</section>
</main>

<footer className="footer">
  <small>Powered by Geolocation API & Open-Meteo (no key required).</small>
</footer>

<style>[]`
:root {
  --bg: #f6f8fb;
}
.dashboard-root {
  color: #111827;
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial;
  padding: 1.5rem;
  min-height: 100vh;
  background: var(--bg);
}
.header h1 {
  margin: 0 0 .25rem;
  font-size: 1.6rem;
}
.tagline {
  margin: 0;
  opacity: .7;
  font-size: .95rem;
}
.grid {
  display: flex;

```

```

317 .grid {
318     display: flex;
319     flex-wrap: wrap;
320     gap: 1.25rem;
321     margin-top: 1rem;
322 }
323 .panel {
324     background: #fff;
325     border-radius: 8px;
326     padding: 1rem;
327     flex: 1 1 260px;
328     min-width: 250px;
329     box-shadow: 0 6px 18px rgba(15,23,42,0.06);
330     display: flex;
331     flex-direction: column;
332     gap: .75rem;
333 }
334 .panel h2 {
335     margin: 0 0 .4rem;
336     font-size: .95rem;
337     text-transform: uppercase;
338     letter-spacing: .4px;
339 }
340 .media .media-wrapper {
341     display: flex;
342     flex-direction: column;
343 }
344 .span-2 {
345     flex: 2 1 520px;
346 }
347 .kv {
348     list-style: none;
349     padding: 0;
350     margin: .25rem 0 .75rem;
351     font-size: .95rem;
352     display: grid;
353     gap: .4rem;
354 }
355 .kv span { opacity: .7; margin-right: .35rem; }
356 .weather > div {
357     display: flex;
358     justify-content: space-between;
359     font-size: .9rem;
360     padding: .15rem 0;
361     border-bottom: 1px dashed rgba(15,23,42,0.06);
362 }
363 .weather > div:last-child { border-bottom: none; }
364 .form {
365     display: grid;
366     grid-template-columns: repeat(auto-fit,minmax(140px,1fr));
367     gap: .5rem;
368     margin-top: .75rem;
369 }
370 .form input {
371     background: #f7fafc;

```

```

369     }
370     .form input {
371       background: #f7fafc;
372       border: 1px solid #e6edf3;
373       color: #0f172a;
374       padding: .5rem .6rem;
375       font-size: .9rem;
376       border-radius: 6px;
377       outline: none;
378     }
379     .form input:focus { border-color: #3da9fc; box-shadow: 0 0 0 3px rgba(61,169,252,0.08); }
380     button {
381       cursor: pointer;
382       border: none;
383       background: linear-gradient(135deg,#3da9fc,#6e45e2);
384       color: #fff;
385       font-weight: 600;
386       font-size: .85rem;
387       padding: .55rem .85rem;
388       border-radius: 8px;
389       box-shadow: 0 6px 18px rgba(46,64,83,0.08);
390     }
391     button:disabled { opacity: .5; cursor: not-allowed; }
392     pre.json {
393       background: #0f172a;
394       color: #e6eef5;
395       border-radius: 8px;
396       padding: .75rem .9rem;
397       max-height: 220px;
398       overflow: auto;
399       font-size: .8rem;
400       margin: .25rem 0 0;
401     }
402     .error { color: #b91c1c; font-size: .85rem; }
403     .muted { opacity: .7; font-size: .9rem; }
404     .footer { text-align: center; font-size: .8rem; opacity: .7; margin-top: 1rem; }
405     @media (max-width: 680px) {
406       .panel { flex: 1 1 100%; }
407       .span-2 { flex: 1 1 100%; }
408       .form { grid-template-columns: repeat(auto-fit,minmax(110px,1fr)); }
409     }
410   </style>
411 </div>
412 )
413 }
414
415 const rootElement = document.getElementById('root')
416 ReactDOM.createRoot(rootElement).render(
417   <StrictMode>
418     <SmartTravelDashboard />
419   </StrictMode>
420 )

```

Smart Travel Companion Dashboard

Your interactive travel assistant

TRAVEL MEDIA



0:00 / 0:01

CURRENT LOCATION

Latitude: 12.8615
Longitude: 77.6648

Get Location

Uses HTML5 Geolocation API.

LIVE WEATHER

Temperature: 27.1°C
Wind Speed: 11.2 km/h
Direction: 303°
Time: 2025-10-06T06:45

TRAVEL PLAN (DYNAMIC JSON)

```
{
  {
    "id": "1",
    "city": "Paris",
    "country": "France",
    "date": "2025-08-11",
    "notes": "Visit Louvre & Eiffel Tower"
  },
  {
    "id": "2",
    "city": "Rome",
    "country": "Italy"
  }
}
```

City Country dd-mm-yyyy

Notes Add Destination

USER DATA (AXIOS FETCH)

Name: Sabrina Graham
Email: Streaming@pl.biz
Phone: +1-770-736-8031 x56442

Fetch Data

BOOKING XML → JSON

Convert Booking XML

Powered by Geolocation API & Open-Meteo (no key required).

Smart Travel Companion Dashboard

Your interactive travel assistant

TRAVEL MEDIA



0:01 / 0:01

CURRENT LOCATION

Latitude: 12.8615
Longitude: 77.6648

Get Location

Uses HTML5 Geolocation API.

LIVE WEATHER

Temperature: 27.1°C
Wind Speed: 11.2 km/h
Direction: 303°
Time: 2025-10-06T06:45

TRAVEL PLAN (DYNAMIC JSON)

```
{
  "country": "Italy",
  "date": "2025-08-15",
  "notes": "Colosseum and Vatican Museums"
},
{
  "id": "1256973253199",
  "city": "Hello",
  "country": "HI",
  "date": "2026-01-11",
  "notes": "go there"
}
}
```

City Country dd-mm-yyyy

Notes Add Destination

USER DATA (AXIOS FETCH)

Name: Leanne Graham
Email: Sincere@april.biz
Phone: 1-770-736-8031 x56442

Fetch Data

BOOKING XML → JSON

Convert Booking XML

```
{
  "booking": {
    "id": "BK12345",
    "traveler": {
      "firstName": "Alice",
      "lastName": "Wong"
    },
    "flights": {
      "flight": [
        {
          "from": "NYC",
          "to": "LHR"
        }
      ]
    }
  }
}
```